# Hack yourself. Then fix it.

Check the app you built with AI,
one feature at a time.



LEARN THE FAILURE

KEEP THE EVIDENCE

VERIFY THE REPAIR

Illustrative risk grid. No real findings shown.

START HERE

# One useful check beats a vague verdict.

Read the boundary, prepare two accounts, and keep the result small enough to explain.

01 **Define the boundary** — 02 **Compare two accounts** — 03 **Repair and retest**

## Choose a private feature

Pick a note, file, invoice, or other record that belongs to one account. Use a staging copy and fictional data. Give the record a harmless marker so you can recognize it.

## Confirm the normal action

Check that the owner can read the record. Without that positive control, a blocked request could simply mean the feature is broken or your setup is wrong.

## Try the forbidden action

With the other ordinary account, request that same record. Inspect the result and the active session. A different organization ID in the address bar does not change which account made the request.

## Make a decision from evidence

A private marker returned to the wrong account is useful evidence. An empty response, failed tool, or HTTP 200 alone does not establish the same result.

## Then widen the review

Use the deeper-review prompt and complete checklist after you have one result you understand. Keep gaps and failed tools visible in your coverage log.

BEFORE TESTING

# Agree the scope. Enforce the limits.

Permission for one app does not include every service it happens to link to.

## Write down the boundary

Use templates/TEST-SCOPE.md: owner, written permission, exact hosts, accounts, actions, test window, request budget, spending controls, and stop conditions. Resolve missing details before the first test request.

## Keep the environment disposable

Use an isolated copy, synthetic records, ordinary test accounts, and a backup. Do not hand the testing agent a production admin or database service-role key.

## Configure the real controls

Constrain the runtime's network, files, and tools. Set enforced service quotas and monitor actual requests. Prompt wording alone does not impose a network boundary, concurrency limit, or spending cap.

## Stop at the boundary

Stop for real private data, unexpected third-party hosts, destructive operations, real charges or messages, or service instability. Preserve minimal redacted evidence and use the agreed contact route.

## Understand the limits

This is education, not legal advice, a professional assessment, or a certification. The full terms are in DISCLAIMER.md. A test login or public URL is not permission to test a third-party system.

SETUP

# Prove that a tool ran.

A model describing a test is different from a model executing one.

01 **Open staging page** — 02 **Read own marker** — 03 **Inspect tool result**

## Use the setup prompt first

prompts/00-prepare-session.txt asks the assistant to confirm scope, available tools, session separation, and any controls that must be configured outside the prompt.

## Verify the account session

Keep account A and B in separate profiles or explicit sessions. Check the signed-in identity before each comparison. An owner token can make an ordinary-looking request misleading.

## Protect the evidence

Use the approved private credential mechanism. Redact passwords, tokens, cookies, reset links, personal details, and private file URLs before sharing. Review the AI provider's handling of submitted data.

## Keep a known working setup

Use the linked model setup kit for DeepSeek Harness connections. This field kit does not install a model, include API credits, or promise any provider will remain available.

## If tools fail

Record an incomplete check. A 401 may mean the test login is invalid; a refused localhost connection may mean the server is off. Neither proves a vulnerability or complete protection.

ACCESS CHECK

# The URL chooses a record. Permission decides who gets it.

Organization and record IDs help locate information. They are not substitutes for authorization.

01 **Owner reads marker** — 02 **Other account asks** — 03 **Server checks access**

## Use a test record with an obvious owner

Add a fictional note to account B and confirm B can read it. Record the allowed sharing rule before making a comparison.

## Compare the actual request

Use account A's session to request B's record through the same route or underlying API. Compare the returned marker, not just the page layout or status code.

## Follow related paths inside scope

A list endpoint, individual record endpoint, search result, export, or copied file link may enforce different rules. Treat each tested action as its own coverage item.

## Fix the authorization layer

Validate the caller's relationship to the record and workspace on the server or database. Hiding the menu, changing an ID format, or blocking one URL is not enough if another data route is still exposed.

## Retest both sides

Repeat A's forbidden read, then B's legitimate read. Record the deployed version and keep a regression check. Source: AUTH in SOURCES.md.

EVERYDAY FEATURE 01

# The password reset that changes the wrong account

A conditional teaching example. This is not a finding about your app or a client.

01 Request for A — 02 Bind token to A — 03 Reject target B

## The normal moment

You forget your password, so you ask the app to email you a reset link. That link should only let you change the password for the account that requested it.

## Where it can go wrong

Imagine the server checks that the link is valid, but then trusts a separate account ID sent by the browser. If it fails to bind both to the same account, a valid link for one test account could change the other account instead.

## A controlled check

Use two disposable accounts and their own inboxes. Request a reset for account A and verify that the reset flow cannot target account B. Check normal reset success, wrong-account rejection, reuse, and expiry within your agreed test scope.

## What to change

Derive the target account from the validated reset token on the server. Use an appropriate expiry and single-use invalidation. Review session handling after a reset.

## Keep the claim honest

Record which account requested the link and which disposable account changed. Redact the link and token. A changed email field on screen is not evidence of a completed password change.

Reference: RESET in SOURCES.md. No reset token, exploit payload, or real account is included. A request error or an invalid link alone does not prove the entire reset flow is safe.

EVERYDAY FEATURE 02

# The review that starts behaving like a page

A conditional teaching example. This is not a finding about your app or a client.

01 Customer review — 02 Safe text display — 03 No page control

## The normal moment

A customer leaves a review, and the next visitor reads it. The review is supposed to be somebody's words, even if you let them add a little formatting.

## Where it can go wrong

If the app inserts that text into an executable browser context without the right handling, the review could change the page or run code. Depending on the app, that could let someone show a fake sign-in form and collect what a visitor types.

## A controlled check

In an isolated copy, use an approved harmless marker to check how review text is rendered. If a browser-execution check is authorized, use a benign local demonstration with no external destination. Do not ask real people to log in to a fake form.

## What to change

Use safe text rendering and context-appropriate output encoding. If formatting is allowed, use a maintained HTML sanitizer with a restrictive policy. Browser security controls are an additional layer.

## Keep the claim honest

Show the stored test input and the browser behavior it caused. Keep script execution, a changed page, and credential theft as separate claims; one does not automatically prove the others.

Reference: XSS in SOURCES.md. This story describes a possible app mistake. A plain-text marker displaying correctly does not by itself prove every input context is safe.

EVERYDAY FEATURE 03

# The chat box that keeps spending your money

A conditional teaching example. This is not a finding about your app or a client.

01 **Chat request** — 02 **Server quota check** — 03 **Then paid call**

## The normal moment

You add an AI chat box so customers can get help without waiting for you. Each useful answer may also create a provider charge.

## Where it can go wrong

If the server accepts unlimited paid requests, one person could repeatedly trigger answers without creating any real business for you. A hidden button or a limit enforced only by the browser would not stop direct requests.

## A controlled check

Set a deliberately small staging allowance. Make only the few agreed requests needed to reach it, then verify that the next request is rejected before the paid provider call. Use a mock provider when possible and inspect server logs.

## What to change

Enforce authentication, per-user quotas, and appropriate rate limits before expensive work. Add the provider-side spending controls available to you and plan what the app does when the budget is exhausted.

## Keep the claim honest

Keep the request count, server decision, and evidence of whether the provider was called. An error shown after the provider already answered may still incur usage.

Reference: REST in SOURCES.md. This is a quota check, not permission for a load test. An alert does not necessarily stop charges; verify the controls your provider actually enforces.

EVERYDAY FEATURE 04

# The checkout that believes the browser's price

A conditional teaching example. This is not a finding about your app or a client.

01 **Trusted price** — 02 **Verified payment** — 03 **Then access**

## The normal moment

You list a workshop for $100. A customer clicks buy, pays, and the app gives them access.

## Where it can go wrong

If the server trusts a price sent by the browser, a customer could ask it to charge $1 instead. The payment provider might correctly process that $1 while your app mistakenly unlocks the $100 workshop.

## A controlled check

Use the payment provider's sandbox or test mode with a fictional product. Verify that the server selects the trusted product price, that changing a browser value cannot reduce the order total, and that access waits for the correct completed payment.

## What to change

Calculate prices and discounts from trusted server-side records. Check payment state, amount, currency, and order association before fulfillment. Verify webhook signatures and handle repeated events safely.

## Keep the claim honest

Record the test product, trusted total, test payment state, and access result. A writable order value does not prove money moved or that paid content was delivered.

Reference: PAYMENTS in SOURCES.md. The fictional dollar amounts are an illustration. This is an application-logic problem, not a claim that the payment provider was compromised. Never run the exercise with real charges.

EVERYDAY FEATURE 05

# The invoice email that gives your assistant a new job

A conditional teaching example. This is not a finding about your app or a client.

01 Read due date — 02 Check tool authority — 03 Approve any send

## The normal moment

You ask your assistant when a supplier's invoice is due, so it opens the email to read the date.

## Where it can go wrong

Someone could put instructions inside that email telling the assistant to send a private document elsewhere. If it mistakes the email for your instructions and has the necessary tools, it might try to send the file while you are waiting for a due date.

## A controlled check

Use a made-up document, a controlled inbox, and a harmless request marker. Ask for the due date and inspect whether the assistant proposes any unrelated tool action. Keep sending disabled or approval-gated during the check.

## What to change

Treat retrieved content as untrusted data, limit the assistant's accessible files and actions, and enforce authorization in the tools. Require meaningful approval for a sensitive send, showing the recipient and exact attachment.

## Keep the claim honest

Distinguish reading an instruction, proposing a tool call, attempting a blocked call, and completing a send. A text claim that a file was sent is not delivery evidence.

Reference: AGENTS in SOURCES.md. No model is assumed to follow the malicious instruction. Rewording a system prompt alone is not a complete boundary for an assistant that can read and send private files.

CHECKLIST A

# The same records can leak elsewhere

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- ☐ A01 Private records: check lists and individual records with two separate accounts, including workspaces and linked invoices. The server must check who can read or change each one.
- ☐ A02 Signed-out access: repeat protected-page and data requests without a login; they should not return private records.
- ☐ A03 Search and exports: keep another account's notes, email addresses and rows out of results and downloaded spreadsheets.
- ☐ A04 File links and storage buckets: a private attachment should stay private when its link is copied to a different account.
- ☐ A05 Admin pages: hiding the menu is not enough. An ordinary account must also be blocked from the page's underlying requests.
- ☐ A06 Profile fields and invitations: only allow fields the user can change; stop users granting roles or inviting people with powers they do not have.
- ☐ A07 Database permissions: enforce access rules on every relevant table; limit what the app's database account can read or change.
- ☐ A08 API responses: return only the fields the page needs. A hidden field in a response is still available to the browser.

Implementation references: AUTH. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHECKLIST B

# Getting in without the right login

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- [ ] B01 Sessions: check expiration, logout and revocation after a compromise. Let users see and end other sessions; closing a browser alone does not prove a session is broken.
- [ ] B02 Password resets: use expiring, single-use links for the intended account. Avoid responses that reveal which email addresses have accounts.
- [ ] B03 Passwords and guessing: remove defaults, use strong password hashing and unique admin passwords, add a second sign-in check, and limit repeated failures.
- [ ] B04 Google or GitHub login: verify allowed return addresses and provider protections. Check account linking and email verification so someone cannot claim another person's identity.
- [ ] B05 Redirect links: keep login links from bouncing people to an unapproved site that could impersonate yours.
- [ ] B06 Session tokens and cookies: validate signatures and expiry, protect signing secrets and reject tampered tokens. Protect cookies from script access where appropriate and send them securely.
- [ ] B07 Cross-site requests and CORS: check whether another website can trigger a sensitive action or read a signed-in user's data. Restrict trusted origins and protect state-changing requests.
- [ ] B08 Clickjacking and browser headers: stop sensitive pages being framed inside another site, and check the relevant browser security headers.

Implementation references: AUTH, RESET, CSRF. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHECKLIST C

# Someone else can spend your usage

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- ☐ C01 Private service keys: keep them out of browser downloads, public code, screenshots and exposed configuration files. A public client key is not automatically a secret.
- ☐ C02 If a secret leaked: replace it, check what it could access and review its usage. Removing the visible copy does not invalidate a stolen key.
- ☐ C03 AI chat, search and email: set enforced per-user limits and check access on the server, not just on the screen.
- ☐ C04 Signup and file uploads: bound request frequency, file sizes and storage so abuse cannot fill your database or storage.
- ☐ C05 Costs: use restricted credentials, quotas and the spending controls the provider actually enforces. An alert alone does not stop spending.
- ☐ C06 Availability: use a small, agreed test load. Cache expensive repeated work where safe, but keep quotas and access checks; caching alone does not prevent abuse.
- ☐ C07 Old test deployments and internal dashboards: check whether they are still public, with weaker protections or default credentials.

Implementation references: REST, SECRETS. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHECKLIST D

# A normal input can become an instruction

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- ☐ D01 Comments and profile text: make sure text is displayed safely. If you allow rich formatting, sanitize it; use browser protections as an additional layer.
- ☐ D02 Search and database filters: keep user input separate from database commands with parameterized queries. An error message alone does not prove a data leak.
- ☐ D03 File uploads: check the actual content, size and allowed type. Store uploads where they cannot execute as server code.
- ☐ D04 Download paths: keep a requested filename from reaching files outside its allowed folder.
- ☐ D05 Link previews and URL imports: limit what the server can fetch so a supplied address cannot reach private internal services.
- ☐ D06 Conversion tools and shell commands: keep user input out of executable commands; use safer libraries and fixed allowed operations.
- ☐ D07 Saved objects: do not rebuild arbitrary executable objects from untrusted serialized data. Validate the format and use safer parsers.

Implementation references: XSS, UPLOADS. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHECKLIST E

# The app can believe the wrong price

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- [ ] E01 Prices, discounts and totals: calculate them on the server and verify the paid amount before delivering the product.
- [ ] E02 Paid features: check the account's entitlement on each protected operation. A payment screen alone does not enforce it.
- [ ] E03 Payment notifications: verify the provider signature and process repeated notifications safely, without granting the same benefit twice.
- [ ] E04 Orders and invoices: check who can read them and who can change amounts or payment details. A writable invoice is not automatically a completed transfer.
- [ ] E05 Sensitive account changes: require appropriate reauthentication before changing payout or recovery details.
- [ ] E06 Card data: use the payment provider's supported checkout; avoid collecting raw card details you do not need.
- [ ] E07 Subscriptions: show the charge, renewal timing and trial terms clearly. Test cancellation, and review the current rules of the stores and payment providers you use.

Implementation references: PAYMENTS, WEBHOOKS. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHECKLIST F

# Your own AI can be another way in

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- ☐ F01 Prompt injection: treat instructions found in documents, emails and websites as untrusted content. Test with harmless markers and a controlled destination.
- ☐ F02 Connected tools: give the agent only the data and actions it needs. Require review before sensitive sends, deletions or financial changes.
- ☐ F03 Separate users: one person's conversation, retrieved documents or agent memory must not turn up in another account.
- ☐ F04 Generated code: review changes to authentication, permissions and secret handling before shipping them.
- ☐ F05 Dependencies: check the libraries the AI installed for known vulnerabilities, suspicious packages and unnecessary permissions; update or remove them as appropriate.
- ☐ F06 AI responses that trigger actions: validate the proposed action in ordinary code before executing it. A confident answer is not permission.

Implementation references: AGENTS, AGENCY, DEPENDENCIES. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHECKLIST G

# Would you notice, and could you recover?

Mark each item confirmed, passed, needs_review, not_tested, or not applicable with a reason.

- ☐ G01 Audit logs and monitoring: record important sign-ins, permission changes and unusual access; send alerts someone will actually review.
- ☐ G02 Sensitive logs and source maps: check what you expose in diagnostics. Keep secrets and private records out of public errors, logs and published debugging files.
- ☐ G03 Backups: keep recoverable copies with separate protection, then test a restore. A backup that has never been restored is an untested recovery plan.
- ☐ G04 Incident response: have a way to pause a compromised feature or revoke its access while you investigate; test who can do that.
- ☐ G05 Sensitive data: collect only what you need and protect it in transit and storage. Encryption does not replace permission checks.
- ☐ G06 Privacy, cookies and retention: match your notices and choices to what the app collects, shares and keeps; get appropriate advice for your market.
- ☐ G07 Package licenses: check the terms of third-party code before distributing it. A security scan does not decide your legal obligations.

Implementation references: SECRETS, DEPENDENCIES. Review against your app's context; these prompts are not a complete audit or automatic tests.

CHOOSE YOUR NEXT CHECK

# Start where a mistake would hurt your business.

These examples help you choose a feature. They do not claim a business has a vulnerability.

### Agency or client portal

Start with private client notes, shared links, attachments, and workspace separation. A different client should not receive a record simply by following a copied link.

### Course, membership, or paid download

Check the path from trusted product price to completed payment to access. Check cancellation and entitlement changes against the rules you actually offer.

### Online shop or booking app

Check order ownership, discounts, invoice details, and confirmations in test mode. Record what the server believes before concluding that a displayed total caused a real payment.

### AI support tool or document assistant

Check per-user usage limits, private retrieval boundaries, and the tools that can send or change data. Use made-up files and an approval-gated sending tool.

### Internal team dashboard

Check roles, exports, account linking, and offboarding. A team member losing a menu should also lose the underlying access that the role no longer allows.

### Use the coverage log

Choose a few high-value checks, record exactly what you covered, and mark the rest untested. The full checklist helps plan the next review without implying you tested everything today.

EVIDENCE

# Write a finding someone can reproduce.

Keep observations, possible consequences, and missing proof separate.

## Identity and expected rule

Record the active account, role, record owner, route, version, and what should have happened. Confirm the record was supposed to be private.

## Smallest proof

Keep the exact action and a redacted response or screenshot that shows the harmless marker or completed result. Do not collect an entire customer database to prove one read.

## Observed impact

State only what the evidence shows. Reading one label is not reading every conversation. A writable field is not a completed financial transfer. An attempted send is not a delivered file.

## Next action

Recommend the specific permission, validation, or business-rule repair. Assign an owner and a retest. Use templates/FINDING.md and templates/RETEST.csv.

## Four report statuses

confirmed: evidence supports the issue. passed: this exact check met the expected rule. needs_review: concern or incomplete proof. not_tested: no executed check. Keep the last two out of the confirmed count.

RISK MAP

# Make the priorities visible.

Qualitative ratings support a decision. They are not CVSS or measured probabilities.

01 Confirm evidence — 02 Rate ease and impact — 03 Choose next repair

## Ease of exploitation, 1 to 5

1 means unusual access or difficult conditions. 3 means an ordinary signed-in user can reproduce the issue with investigation. 5 means a public, repeatable action with few barriers. Explain the conditions.

## Consequence, 1 to 5

1 means limited nuisance or information. 3 means meaningful private data or a workflow effect. 5 means broad compromise or severe impact supported by evidence. Do not rate a hypothetical worst case as an observed result.

## A simple triage default

The urgent list includes confirmed items with both ratings at least 3, plus documented owner overrides. The grid uses ease times consequence for color. These are kit-specific display choices, not an industry severity standard.

## Use the offline report

Open report/risk-report.html and import a redacted JSON file based on templates/assessment.blank.json. The sample is clearly fictional. The page runs locally and starts without findings; no scan is performed.

## Export before closing

The page keeps data in memory, not persistent storage. Export the JSON to retain it, and use Print / Save PDF for a shareable view. Check the content before sharing either file. Full anchors: guides/RISK-RATINGS.md.

REPAIR AND RECOVERY

# A fix needs two successful checks.

The forbidden action should fail, and the legitimate action should still work.

01 **Review the repair** — 02 **Repeat failed boundary** — 03 **Confirm owner access**

## Apply a reviewed change in staging

Use the evidence to identify the rule that failed. Review the affected files, likely side effects, and rollback plan. Keep unrelated changes out of the repair.

## Repeat the same input

Use the original account, route, record, and action. A different request may miss the original problem. Record the response and version, then confirm the owner can still complete the normal action.

## Check the neighboring routes

Verify related list, item, export, file, and write paths affected by the change. Retain an appropriate regression test. A local pass does not prove production has the same deployed repair.

## Clean up and revoke

Remove disposable records and test access, revoke temporary credentials, and keep only necessary redacted evidence. Restore changes according to the scope and recovery plan.

## Escalate a real exposure

Stop and use the authorized owner's incident process. Seek qualified help for material risk or sensitive production data. This guide does not determine your legal or regulatory duties.

PROMPTS AND FILES

# The next step is already in the kit.

Use one prompt for one job. Keep your scope and evidence beside it.

### 00 / Prepare the session

prompts/00-prepare-session.txt - confirm scope, tools, sessions, and controls before testing.

### 01 / Run one private-record check

prompts/01-first-check.txt - confirm owner access, then test the other account.

### 02 / Go deeper

prompts/02-go-deeper.txt - map related in-scope routes with a counted request budget.

### 03 / Review the evidence

prompts/03-review-evidence.txt - challenge assumptions before calling something confirmed.

### 04 / Fix and retest

prompts/04-fix-and-retest.txt - review a repair, apply it when authorized, and verify both sides.

### 05 / Build the risk report

prompts/05-build-risk-report.txt - prepare redacted JSON for the offline heat map.

### Keep the complete sources

ALL-PROMPTS.md combines the prompts. CHECKLIST.md, templates/, and guides/ are editable. DISCLAIMER.md and DEFENSIVE-USE.md explain the full boundaries. The PDFs are finished reading copies.

READ THIS BEFORE USE

# Know what this kit can and cannot tell you.

Full notice: DISCLAIMER.md. Third-party terms and licenses still apply.

## Authorization

Use only systems you own or have explicit written permission to test, within the agreed scope. The kit grants no permission. A provider account, test login, or public URL is not authorization for unrelated systems.

## No guarantee

Provided as is and as available, without warranties to the extent permitted by applicable law. This is education, not legal advice, a professional security assessment, compliance certification, or a guarantee of security or fitness. AI can be incorrect or unsafe.

## Your responsibilities

Review commands, findings, code, costs, data handling, permissions, backups, and third-party terms. Use synthetic data and limited access. A prompt is not a sandbox, and a passing check covers only the action tested.

## Liability and rights

To the extent permitted by applicable law, creators and contributors disclaim liability for losses arising from use or reliance. Nothing excludes rights, remedies, warranties, or liability that cannot lawfully be excluded or limited.

## Examples and providers

Feature stories and sample reports are fictional teaching examples. No client data, model weights, credits, or actual app findings are included. Product names belong to their owners; references do not imply endorsement.

## Version and verification

Version 1.0.0, 29 October 2026. See verification/STATUS.md for the actual checks and remaining limits. Provider interfaces, models, prices, and guidance can change.

REFERENCES AND UPDATES

# Keep the official guidance close.

Use these sources for implementation details and current provider behavior.

### AUTH / Authorization
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

### RESET / Forgot Password
https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html

### XSS / Cross Site Scripting Prevention
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

### REST / REST Security
https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html

### SECRETS / Secrets Management
https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

### AGENTS / AI Agent Security
https://cheatsheetseries.owasp.org/cheatsheets/AI_Agent_Security_Cheat_Sheet.html

### AGENCY / Excessive Agency
https://genai.owasp.org/llmrisk/llm062025-excessive-agency/

### PAYMENTS / Stripe: Fulfill orders
https://docs.stripe.com/checkout/fulfillment

### WEBHOOKS / Stripe: Manage webhook endpoints
https://docs.stripe.com/events/manage-webhook-endpoints

### UPLOADS / File Upload
https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

### CSRF / Cross-Site Request Forgery Prevention
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

### DEPENDENCIES / Vulnerable Dependency Management
https://cheatsheetseries.owasp.org/cheatsheets/Vulnerable_Dependency_Management_Cheat_Sheet.html

Updated sources and editable files: github.com/earlyaidopters/hack-yourself-security-kit
Complete download: markkashef.gumroad.com/l/hack-yourself-checklist